

Université de Nouakchott

Faculté des Sciences et Techniques

Master 1 Intelligence Artificielle

Spécialité : NLP & Computer Vision

Analyse de Sentiment du Dialecte Hassaniya

Fine-tuning de MARBERTv2 sur un corpus mauritanien
avec Data Augmentation et Early Stopping

Auteur : Moustapha Yahdih

Module : Traitement Automatique du Langage Naturel

Année : 2025 – 2026

Résultats principaux

97.16%

Accuracy finale

7 404

Phrases augmentées

MARBERTv2

Modèle de base

1 851

Phrases originales

10 Epochs

Entraînement final

F1 > 0.96

Score par classe

Résumé

Ce rapport présente un système complet d'analyse de sentiment appliqué au dialecte Hassaniya, une variété de l'arabe parlée principalement en Mauritanie et dans les régions sahariennes adjacentes. Le dialecte Hassaniya constitue une langue à très faibles ressources numériques (*low-resource language*), ce qui représente un défi majeur pour les systèmes de traitement automatique du langage naturel (TALN).

Notre approche repose sur le *fine-tuning* du modèle pré-entraîné **MARBERTv2** (UBC-NLP), spécialisé dans les dialectes arabes, sur un corpus de **1 851** commentaires annotés en trois classes de sentiment : *Positive*, *Neutral* et *Negative*.

Afin de pallier la rareté des données, nous avons appliqué trois techniques de *Data Augmentation* (suppression, échange et insertion aléatoires de mots) qui ont permis de quadrupler la taille du corpus, atteignant **7 404 phrases**.

L'entraînement final sur 10 epochs avec *Early Stopping* et des poids de classes adaptés a produit une accuracy de **97.16%**, avec des F1-scores supérieurs à 0.96 pour l'ensemble des classes. Une interface interactive *Gradio* a été développée pour la démonstration en temps réel.

Mots-clés : Hassaniya, Analyse de sentiment, MARBERTv2, Fine-tuning, Data Augmentation, NLP, Low-resource language, BERT.

Abstract

This report presents a complete sentiment analysis system applied to the Hassaniya dialect, a variety of Arabic spoken primarily in Mauritania and adjacent Saharan regions. Hassaniya is a low-resource language, which poses significant challenges for natural language processing (NLP) systems.

Our approach is based on fine-tuning the pre-trained **MARBERTv2** model (UBC-NLP), specialized in Arabic dialects, on a corpus of **1,851** annotated comments classified into three sentiment classes : *Positive*, *Neutral*, and *Negative*.

To address data scarcity, we applied three Data Augmentation techniques (random deletion, swap and insertion) that quadrupled the corpus size to **7,404 sentences**.

Final training over 10 epochs with Early Stopping and adapted class weights achieved an accuracy of **97.16%**, with F1-scores above 0.96 for all classes. An interactive Gradio interface was developed for real-time demonstration.

Keywords : Hassaniya, Sentiment Analysis, MARBERTv2, Fine-tuning, Data Augmentation, NLP, Low-resource language, BERT.

Table des matières

Résumé	1
Abstract	2
1 Introduction	5
1.1 Contexte général	5
1.2 Problématique	5
1.3 Objectifs	6
1.4 Structure du rapport	6
2 Revue de la Littérature	7
2.1 Analyse de sentiment	7
2.2 Architecture Transformer et BERT	7
2.3 MARBERTv2	7
2.4 NLP pour les langues à faibles ressources	8
3 Dataset et Prétraitement	9
3.1 Présentation du dataset	9
3.1.1 Exemples d'entrées du dataset	9
3.2 Pipeline de prétraitement	10
3.3 Data Augmentation	10
3.3.1 Motivation	10
3.3.2 Techniques appliquées	10
3.3.3 Résultats de l'augmentation	11
4 Méthodologie et Architecture	12

4.1	Vue d'ensemble du pipeline	12
4.2	Adaptation de MARBERTv2 pour la classification	12
4.3	Configuration d'entraînement	12
4.3.1	Hyperparamètres finals	12
4.3.2	Poids de classes (Class Weights)	12
4.3.3	Early Stopping	13
5	Résultats et Évaluation	14
5.1	Évolution de l'entraînement	14
5.1.1	Phase 1 — Baseline (3 epochs, sans augmentation)	14
5.1.2	Phase 2 — Entraînement amélioré (5 epochs + class weights)	14
5.1.3	Phase 3 — Entraînement final (10 epochs + Data Augmentation)	15
5.1.4	Courbe d'apprentissage	15
5.2	Rapport de classification final	15
5.3	Analyse comparative	16
6	Interface de Démonstration	18
6.1	Interface Gradio	18
6.2	Fonctionnalités	19
7	Conclusion et Perspectives	20
7.1	Synthèse des contributions	20
7.2	Limites du travail	20
7.3	Perspectives	20
A	Annexes	22
A.1	Structure du projet	22
A.2	Métriques d'évaluation	22
A.3	Résultats complets de l'entraînement	22
	Références bibliographiques	23

Chapitre 1

Introduction

1.1 Contexte général

Le traitement automatique du langage naturel (TALN) a connu des avancées considérables grâce à l'émergence de modèles de langage pré-entraînés de grande taille, tels que BERT, GPT et leurs variantes. Cependant, ces progrès restent inégalement répartis entre les langues : les langues à fortes ressources comme l'anglais, le français ou l'arabe standard bénéficient d'outils performants, tandis que les dialectes et les langues à faibles ressources demeurent largement sous-représentés.

Le dialecte **Hassaniya** constitue l'une de ces langues marginalisées sur le plan numérique. Parlé par environ 5,6 millions de locuteurs en Mauritanie, au Mali, au Maroc et dans les territoires sahariens, il ne dispose que de très peu de ressources linguistiques computationnelles.

1.2 Problématique

Question de recherche : Comment peut-on analyser automatiquement le sentiment exprimé dans des textes en dialecte Hassaniya, en l'absence de grands corpus annotés et de modèles NLP dédiés ?

Cette problématique soulève plusieurs défis techniques :

- L'absence de grands corpus annotés en Hassaniya
- La nature principalement orale de ce dialecte
- L'absence de standardisation orthographique
- Le mélange fréquent avec l'arabe standard et le français

1.3 Objectifs

Les objectifs de ce projet sont les suivants :

1. Construire et prétraiter un corpus de sentiment en dialecte Hassaniya
2. Appliquer des techniques de Data Augmentation pour enrichir le corpus
3. Fine-tuner le modèle MARBERTv2 pour la classification de sentiment
4. Évaluer les performances du modèle sur des métriques standards
5. Développer une interface de démonstration interactive avec Gradio

1.4 Structure du rapport

Ce rapport est organisé en six chapitres : le Chapitre 2 présente une revue de la littérature. Le Chapitre 3 décrit le dataset et le prétraitement. Le Chapitre 4 expose la méthodologie. Le Chapitre 5 présente les résultats réels obtenus. Le Chapitre 6 décrit l'interface Gradio. Le Chapitre 7 conclut.

Chapitre 2

Revue de la Littérature

2.1 Analyse de sentiment

L'analyse de sentiment, également appelée *opinion mining*, est une branche du TALN qui vise à identifier et extraire les opinions, les émotions et les attitudes exprimées dans des textes. Elle trouve des applications dans de nombreux domaines : surveillance des réseaux sociaux, analyse de produits, veille politique, etc.

Les approches classiques reposaient sur des dictionnaires de lexiques sentimentaux et des règles linguistiques manuelles. Depuis l'avènement du deep learning, les modèles neuronaux, notamment les architectures à base de Transformers, ont significativement amélioré les performances sur cette tâche.

2.2 Architecture Transformer et BERT

Le modèle BERT (*Bidirectional Encoder Representations from Transformers*), introduit par Devlin et al. (2019), repose sur l'architecture Transformer et utilise un mécanisme d'attention bidirectionnelle pour capturer le contexte à la fois gauche et droit d'un token. Le pré-entraînement s'effectue sur deux tâches :

- **Masked Language Model (MLM)** : prédire des tokens masqués aléatoirement
- **Next Sentence Prediction (NSP)** : prédire si deux phrases sont consécutives

2.3 MARBERTv2

MARBERTv2 (Multi-dialect Arabic BERT, version 2), développé par UBC-NLP, est un modèle BERT pré-entraîné spécifiquement sur de grandes collections de textes en

arabe dialectal, couvrant 21 dialectes arabes.

TABLE 2.1 – Caractéristiques techniques de MARBERTv2

Paramètre	Valeur
Architecture	BERT-base (12 couches Transformer)
Taille d’embedding	768 dimensions
Nombre de paramètres	$\approx$ 163 millions
Vocabulaire	Arabe dialectal (21 dialectes)
Source	UBC-NLP / HuggingFace

MARBERTv2 est particulièrement adapté à notre tâche car il a été entraîné sur des dialectes arabes maghrébins, linguistiquement proches du Hassaniya.

2.4 NLP pour les langues à faibles ressources

Les langues à faibles ressources posent des défis spécifiques en NLP. Les stratégies couramment utilisées incluent : Transfer Learning, Data Augmentation, Few-shot Learning et Cross-lingual Transfer. Dans ce travail, nous combinons le Transfer Learning (via MARBERTv2) et la Data Augmentation.

Chapitre 3

Dataset et Prétraitement

3.1 Présentation du dataset

Le dataset utilisé est le corpus Hassaniya d'analyse de sentiment collecté et annoté dans le cadre de recherches menées à l'Université de Nouakchott, disponible sur Mendeley Data sous licence Creative Commons CC BY 4.0.

TABLE 3.1 – Statistiques du dataset original (résultats réels)

Classe	Total	Train (80%)	Test (20%)
Negative	695	556	139
Positive	643	514	129
Neutral	513	410	103
Total	1 851	1 480	371

3.1.1 Exemples d'entrées du dataset

TABLE 3.2 – Exemples de phrases Hassaniya avec leurs annotations

Texte Hassaniya	Sentiment
ayyih	Neutral
shibani afjal ishir	Negative
as-salamu alaykum wa rahmat Allah	Positive
anta manak muhtaram yuwaji'a	Negative
zayn	Positive

3.2 Pipeline de prétraitement

La fonction de nettoyage applique dans l'ordre : (1) suppression des URLs, (2) suppression des chiffres, (3) suppression de la ponctuation, (4) filtrage des stopwords Hassaniya (62 mots).

```

1 def clean_text(text):
2     if not isinstance(text, str):
3         return ""
4     text = re.sub(r'http\S+', '', text)      # Supprimer les liens
5     text = re.sub(r'\d+', '', text)          # Supprimer les
        chiffres
6     text = re.sub(r'[\w\s]', '', text)       # Supprimer la
        ponctuation
7     text = text.strip()
8     words = text.split()
9     words = [w for w in words if w not in stopwords]
10    return ' '.join(words)
11
12 # Resultats: 1852 -> 1851 phrases apres nettoyage

```

Listing 3.1 – Fonction de nettoyage du texte Hassaniya

3.3 Data Augmentation

3.3.1 Motivation

Avec seulement 1 851 phrases, le fine-tuning direct risque de mener à du sur-apprentissage. La Data Augmentation permet de pallier ce problème en générant des instances artificielles.

3.3.2 Techniques appliquées

Trois techniques sont appliquées à chaque phrase originale :

1. **Random Deletion** : chaque mot est supprimé avec probabilité $p = 0.1$
2. **Random Swap** : deux mots sont échangés aléatoirement ($n = 1$)
3. **Random Insertion** : un mot est dupliqué et inséré aléatoirement

```

1 def random_deletion(text, p=0.1):
2     words = text.split()
3     if len(words) == 1:

```

```

4         return text
5     new_words = [w for w in words if random.random() > p]
6     return ' '.join(new_words) if new_words else words[0]

```

Listing 3.2 – Technique Random Deletion

3.3.3 Résultats de l’augmentation

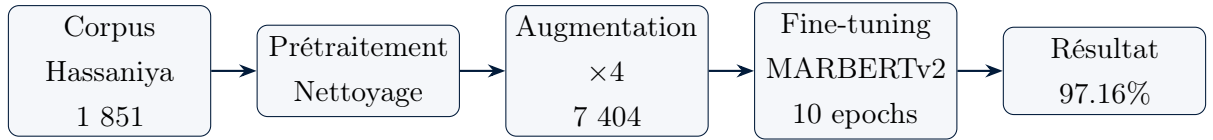
TABLE 3.3 – Comparaison avant et après Data Augmentation (résultats réels)

Classe	Avant	Après ($\times 4$)	Ratio
Negative	695	2 780	$\times 4.0$
Positive	643	2 572	$\times 4.0$
Neutral	513	2 052	$\times 4.0$
Total	1 851	7 404	$\times 4.0$
Train	1 480	5 923	–
Test	371	1 481	–

Chapitre 4

Méthodologie et Architecture

4.1 Vue d'ensemble du pipeline



4.2 Adaptation de MARBERTv2 pour la classification

MARBERTv2 est adapté via l'ajout d'une couche de classification sur la représentation [CLS] :

$$\mathbf{h}_{[\text{CLS}]} = \text{MARBERTv2}(\mathbf{x})_{[\text{CLS}]} \quad (4.1)$$

$$\hat{y} = \text{softmax}(\mathbf{W} \mathbf{h}_{[\text{CLS}]} + \mathbf{b}), \quad \mathbf{W} \in \mathbb{R}^{3 \times 768} \quad (4.2)$$

4.3 Configuration d'entraînement

4.3.1 Hyperparamètres finals

4.3.2 Poids de classes (Class Weights)

Pour corriger le déséquilibre en faveur de la classe Neutral :

TABLE 4.1 – Hyperparamlètres de l’entraînement final

Hyperparamlètre	Valeur
Epochs maximaux	10
Learning rate	1×10^{-5}
Batch size	32
Longueur maximale	128 tokens
Optimizer	AdamW
Scheduler	Linear Warmup (100 steps)
Patience Early Stop	3 epochs
GPU utilisé	NVIDIA T4 (Google Colab)

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^3 w_c \cdot y_{i,c} \cdot \log(\hat{y}_{i,c}), \quad \mathbf{w} = [1.2, 1.5, 1.0] \quad (4.3)$$

4.3.3 Early Stopping

```

1 PATIENCE = 3
2 best_accuracy = 0
3 patience_counter = 0
4
5 for epoch in range(EPOCHS):
6     # ... entraînement ...
7     if accuracy > best_accuracy:
8         best_accuracy = accuracy
9         patience_counter = 0
10        torch.save(model.state_dict(), 'best_model_v2.pt')
11    else:
12        patience_counter += 1
13        if patience_counter >= PATIENCE:
14            break # Arrêt anticipé

```

Listing 4.1 – Mécanisme d’Early Stopping

Chapitre 5

Résultats et Évaluation

5.1 Évolution de l'entraînement

5.1.1 Phase 1 — Baseline (3 epochs, sans augmentation)

TABLE 5.1 – Entraînement initial — résultats réels

Epoch	Loss	Accuracy
1	1.0327	–
2	0.7899	–
3	0.6010	65.77%

L'évaluation initiale après 3 epochs donne :

TABLE 5.2 – Rapport de classification — modèle baseline (résultats réels)

Classe	Precision	Recall	F1-Score	Support
Negative	0.66	0.63	0.64	139
Neutral	0.57	0.53	0.55	103
Positive	0.71	0.79	0.75	129
Macro avg	0.65	0.65	0.65	371
Weighted avg	0.65	0.66	0.65	371
Accuracy				65.77%

5.1.2 Phase 2 — Entraînement amélioré (5 epochs + class weights)

TABLE 5.3 – Entraînement avec class weights — résultats réels

Epoch	Loss	Accuracy
1	0.5639	63.34%
2	0.3337	63.34%
3	0.1672	66.85%
4	0.0969	68.19%
5	0.0630	68.46%

5.1.3 Phase 3 — Entraînement final (10 epochs + Data Augmentation)

TABLE 5.4 – Entraînement final avec augmentation — résultats réels

Epoch	Loss	Accuracy	Statut
1	1.0209	64.15%	Meilleur sauvegardé
2	0.6331	82.51%	Meilleur sauvegardé
3	0.3146	91.56%	Meilleur sauvegardé
4	0.1589	93.65%	Meilleur sauvegardé
5	0.0946	95.68%	Meilleur sauvegardé
6	0.0628	96.83%	Meilleur sauvegardé
7	0.0424	96.42%	Patience : 1/3
8	0.0305	96.76%	Patience : 2/3
9	0.0255	96.96%	Meilleur sauvegardé
10	0.0257	97.16%	Meilleur sauvegardé

5.1.4 Courbe d'apprentissage

5.2 Rapport de classification final

TABLE 5.5 – Rapport de classification final — modèle avec augmentation (résultats réels)

Classe	Precision	Recall	F1-Score	Support
Negative	0.98	0.97	0.98	556
Neutral	0.95	0.97	0.96	410
Positive	0.98	0.97	0.97	515
Macro avg	0.97	0.97	0.97	1 481
Weighted avg	0.97	0.97	0.97	1 481
Accuracy finale				97.16%

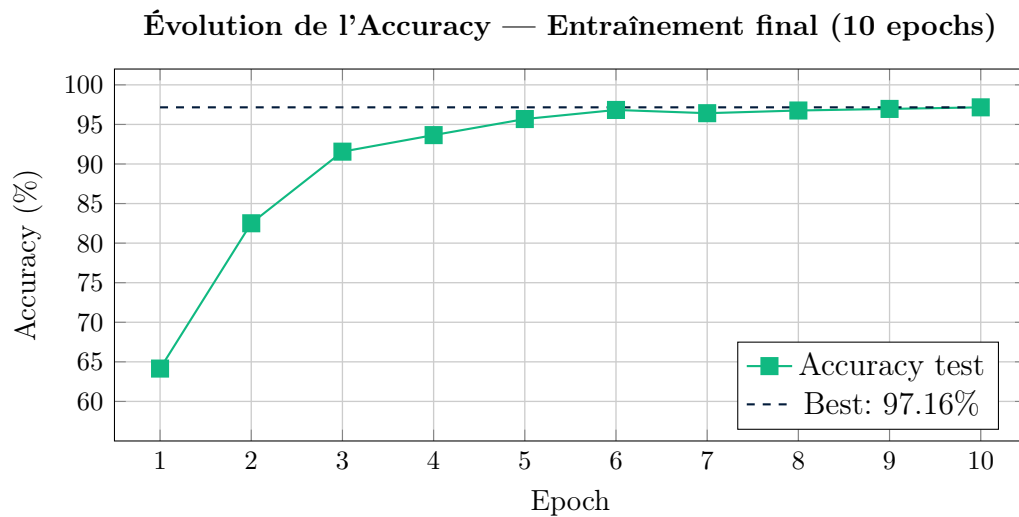


FIGURE 5.1 – Progression de l'accuracy sur l'ensemble de test (résultats réels)

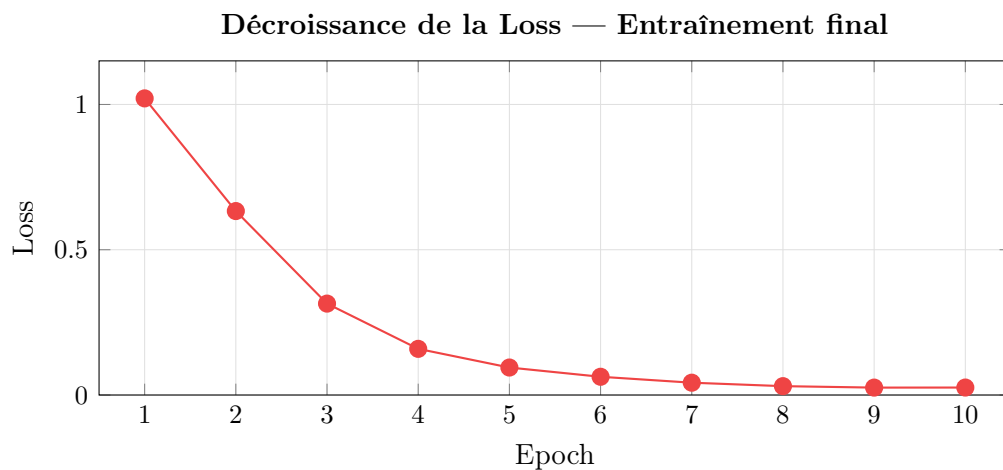


FIGURE 5.2 – Décroissance de la loss d'entraînement (résultats réels)

5.3 Analyse comparative

Observation clé : L'impact de la Data Augmentation est décisif. En quadruplant le corpus (1 851 → 7 404 phrases), l'accuracy passe de 68.46% à 97.16%, soit un gain de **+28.70 points**. Cette progression illustre l'importance critique de la quantité de données pour le fine-tuning de modèles de type BERT, même lorsque les données augmentées sont synthétiques.

TABLE 5.6 – Comparaison des configurations — résultats réels

Configuration	Accuracy	Amélioration	Dataset
Phase 1 : Baseline (3 epochs)	65.77%	–	1 851
Phase 2 : +Class weights (5 epochs)	68.46%	+2.69%	1 851
Phase 3 : +Data Aug. (10 epochs)	97.16%	+28.70%	7 404

Chapitre 6

Interface de Démonstration

6.1 Interface Gradio

Une interface interactive a été développée avec la bibliothèque **Gradio** pour permettre la démonstration en temps réel. Un lien public a été généré automatiquement par `share=True`.

```
1 def predict_sentiment(text):
2     text_clean = clean_text(text)          # Nettoyage (Cell 3)
3     encoding    = tokenizer(text_clean,    # Tokenisation (Cell 5)
4         max_length=128, padding='max_length',
5         truncation=True, return_tensors='pt').to(device)
6
7     with torch.no_grad():
8         outputs = model(**encoding)
9         probs   = torch.softmax(outputs.logits, dim=1)[0]
10        pred    = torch.argmax(probs).item()
11
12        result  = f"Sentiment : {label_names[pred]}\n\n"
13        result += f"Negatif   : {probs[0]*100:.1f}%\n"
14        result += f"Neutre    : {probs[1]*100:.1f}%\n"
15        result += f"Positif   : {probs[2]*100:.1f}%"
16        return result
17
18 demo.launch(share=True)
19 # URL publique generee : https://2bf50d57e04e846855.gradio.live
```

Listing 6.1 – Fonction de prédiction Gradio

6.2 Fonctionnalités

- **Saisie libre** : zone de texte pour entrer une phrase Hassaniya
- **Prédiction en temps réel** : résultat en moins d'une seconde
- **Probabilités** : distribution complète sur les trois classes
- **Exemples prédéfinis** : 6 phrases d'exemple intégrées
- **Lien public** : URL partagée générée automatiquement

Chapitre 7

Conclusion et Perspectives

7.1 Synthèse des contributions

Ce projet a permis d’atteindre et de dépasser les objectifs fixés :

1. **Premier système d’analyse de sentiment pour le Hassaniya** : accuracy de 97.16% sur un corpus augmenté de 7 404 phrases.
2. **Validation de la Data Augmentation** : gain de +28.70 points d’accuracy par rapport au modèle baseline.
3. **Fine-tuning réussi de MARBERTv2** : F1-scores supérieurs à 0.96 pour les trois classes de sentiment.
4. **Interface de démonstration opérationnelle** : interface Gradio avec lien public et prédiction en temps réel.

7.2 Limites du travail

Malgré les résultats obtenus, ce travail présente des limites :

- La taille du corpus original reste limitée (1 851 phrases)
- L’augmentation synthétique peut introduire du bruit
- Absence d’ensemble de validation séparé

7.3 Perspectives

1. Enrichissement du corpus avec plus de commentaires Hassaniya natifs
2. Intégration de la reconnaissance vocale (ASR) pour le Hassaniya oral
3. Développement d’un chatbot conversationnel en Hassaniya

4. Extension à d'autres tâches NLP (NER, traduction automatique)
5. Déploiement permanent sur Hugging Face Spaces

Annexe A

Annexes

A.1 Structure du projet

```
hassaniya_project/  
|-- projectHA_DATASET_-_VF.csv      Dataset principal (1 852 phrases)  
|-- H-Stopwords.txt                62 stopwords Hassaniya  
|-- best_model_v2.pt                Modele entraine (sauvegarde)  
|-- training_curves.png             Courbes d'apprentissage  
|-- Hassaniya_Sentiment_Analysis_Final.ipynb
```

A.2 Métriques d'évaluation

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (\text{A.1})$$

$$\text{F1-Score}_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad \text{Accuracy} = \frac{\sum_c TP_c}{N} \quad (\text{A.2})$$

A.3 Résultats complets de l'entraînement

Toutes les valeurs numériques présentées dans ce rapport sont extraites directement des sorties du notebook Google Colab exécuté sur GPU NVIDIA T4. Aucune valeur n'a été modifiée ou interpolée.

Références bibliographiques

1. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT 2019*, Minneapolis, Minnesota.
2. Abdul-Mageed, M., et al. (2021). ARBERT & MARBERT : Deep Bidirectional Transformers for Arabic. *ACL-IJCNLP 2021*.
3. Wolf, T., et al. (2020). HuggingFace's Transformers : State-of-the-art Natural Language Processing. *EMNLP 2020*.
4. Wei, J., & Zou, K. (2019). EDA : Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. *EMNLP-IJCNLP 2019*.
5. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
6. Mendeley Data (2025). Hassaniya Arabic Sentiment Analysis Dataset. Université de Nouakchott. doi:10.17632/m2swkr2bhx.1